

Non-overlapping Intervals

Problem Statement: Given an array of N intervals in the form of (start[i], end[i]), where start[i] is the starting point of the interval and end[i] is the ending point of the interval, return the minimum number of intervals that need to be removed to make the remaining intervals non-overlapping. .

Examples

Input: Intervals = [[1, 2], [2, 3], [3, 4], [1, 3]]

Output: 1

Explanation: You can remove the interval [1, 3] to make the remaining intervals non-overlapping.

Input: Intervals = [[1, 3], [1, 4], [3, 5], [3, 4], [4, 5]]

Output: 2

Explanation: You can remove the intervals [1, 4] and [3, 5] to make the rest non-overlapping.

Brute force Approach

Algorithm

We try all possible combinations by removing one interval at a time and check if the remaining intervals are non-overlapping. In brute force, we consider every subset of intervals and check if it's valid (non-overlapping). Then we keep track of the max size of such valid subset. $\text{answer} = \text{total intervals} - \text{max valid subset size}$.

- Generate all subsets of the given intervals.
- For each subset, check if the intervals are non-overlapping by sorting and comparing end times.
- Track the size of the largest valid subset (non-overlapping).
- Return total intervals - size of max valid subset.

Intervals = {(1, 2), (1, 3), (3, 4)} n = 3

Bitmasking

- mask = 000 --> pick nothing
subset = { }
valid, size = 0 (maxValid = 0)
- mask = 001 --> pick {1, 2}
subset = { (1, 2) }
valid, size = 1 (maxValid = 1)
- mask = 010 --> pick {1, 3}
subset = { (1, 3) }
valid, size = 1 (maxValid = 1)
- mask = 011 --> pick {1, 2}, {1, 3}
sort : { (1, 2), (1, 3) }
check : $1 < 2$? Yes, Hence overlap (invalid)
- mask = 100 --> pick {3, 4}
subset = { (3, 4) }
valid, size = 1 (maxValid = 1)

→ **mask = 101** --> pick {1, 2}, {3, 4}
 sort : { (1, 2), (3, 4) }
 check : 3 < 2 ? No, size = 2 (maxValid = 2)

→ **mask = 110** --> pick {1, 3}, {3, 4}
 sort : { {1, 3}, {3, 4} }
 check : 3 < 3 ? **No --> Valid, size = 2 (maxValid = 2)**

→ **mask = 111** --> pick all three (1, 2), (1, 3), (3, 4}
 sort : { (1, 2), (1, 3), (3, 4) }
 check : First two overlap (1<2) --> **invalid**

maxValid = 2

answer = n - maxValid
 = 3 - 2
 = 1

Code

```

import java.util.*;

class Solution {
    // Function to find the minimum number of intervals to remove to make all
    intervals non-overlapping
    public int eraseOverlapIntervals(int[][] intervals) {

        // Total number of intervals
        int n = intervals.length;

        // Track max valid non-overlapping subset
        int maxValid = 0;

        // Try all subsets using bitmasking
        for (int mask = 0; mask < (1 << n); mask++) {

            // Build the subset
            List<int[]> subset = new ArrayList<>();
            for (int i = 0; i < n; i++) {
                if ((mask & (1 << i)) != 0)
                    subset.add(intervals[i]);
            }

            // Sort subset by start time
            subset.sort((a, b) -> Integer.compare(a[0], b[0]));

            // Check for overlapping
            boolean valid = true;
            for (int i = 1; i < subset.size(); i++) {
                if (subset.get(i)[0] < subset.get(i - 1)[1]) {
                    valid = false;
                    break;
                }
            }
        }
    }
}

```

```

        // Update max valid size
        if (valid)
            maxValid = Math.max(maxValid, subset.size());
    }

    // Return total - max valid
    return n - maxValid;
}
}
// Driver Code
public class Main {
    public static void main(String[] args) {
        Solution sol = new Solution();
        int[][] intervals = { {1, 2}, {2, 3}, {3, 4}, {1, 3} };
        System.out.println("Minimum intervals to remove: " +
sol.eraseOverlapIntervals(intervals));
    }
}

```

Complexity Analysis

Time Complexity: $O(2^N \times N \log N)$ The total number of subsets of N intervals is 2^N , and for each subset, we sort the subset, which takes $O(N \log N)$. We check if the intervals in the subset are non-overlapping, which takes $O(N)$.

Space Complexity: $O(N)$, We use extra space to store subsets of intervals, which can hold up to N intervals per subset.

Optimal Approach

Algorithm

To keep the maximum number of non-overlapping intervals, we need to remove the minimum number of overlapping ones. This is a classic greedy scenario. We sort intervals by their end times and always pick the next interval that starts after the last selected one ends. The rest need to be removed.

- Sort all intervals based on their end time in ascending order.
- Initialize a variable to store the end time of the last non-overlapping interval (e.g., set to the end of the first interval).
- Initialize a counter to track how many intervals need to be removed.
- Traverse the remaining intervals:
- If the start of the current interval is greater than or equal to the end time of the last added interval, update the end time.
- Else, it overlaps increment the removal counter.
- Return the counter at the end as the minimum number of intervals to remove.

Intervals = { (1, 3), (2, 4), (3, 5), (1, 2) }

Sort by end time (Second value in each pair)

{ (1, 2), (1, 3), (2, 4), (3, 5) }

→ **Initialize** : count = 0 (as no intervals removed yet)
prevEnd = 2 (end time of the first interval {1, 2})

→ **Traversing** : i = 1 {1, 3}
check if it start before prevEnd?
start = 1
preEnd = 2 --> 1 < 2 --> **Yes, overlap**
Remove this interval (count = 1)
preEnds stays = 2 (because we keep the earlier ending interval)

→ **i = 2 interval** {2, 4}
check if 2 < 2 ? No, because no overlap, keep it update prevEnd = 4

→ **i = 3 interval** {3, 5}
check if 3 < 4 ? Yes, overlap, remove it, count = 2
prevEnd stays = 4

End of Loop : count = 2
(That's min no of interval to remove to avoid overlap)

Code

```
import java.util.*;

class Solution {
    // Function to find the minimum number of intervals to remove to make all
    intervals non-overlapping
    public int eraseOverlapIntervals(int[][] intervals) {
        // Sort intervals by their end time
        Arrays.sort(intervals, (a, b) -> a[1] - b[1]);

        // Counter for how many intervals we need to remove
        int count = 0;

        // Track end time of last non-overlapping interval
        int prevEnd = intervals[0][1];

        // Loop through remaining intervals
        for (int i = 1; i < intervals.length; i++) {
            // If current interval overlaps with previous
            if (intervals[i][0] < prevEnd) {
                // Increment count to remove this interval
                count++;
            } else {
                // Update previous end time
                prevEnd = intervals[i][1];
            }
        }

        // Return total number of intervals removed
        return count;
    }
}
```

```

}

// Main class
public class Main {
    public static void main(String[] args) {
        Solution sol = new Solution();

        // Sample input
        int[][] intervals = { {1, 3}, {2, 4}, {3, 5}, {1, 2} };

        // Output result
        System.out.println("Minimum number of intervals to remove: " +
sol.eraseOverlapIntervals(intervals));
    }
}

```

Complexity Analysis

Time Complexity: $O(n \log n)$, This is because we sort the intervals by their end time, which takes $O(n \log n)$ time. Then we traverse the sorted intervals once, which takes $O(n)$ time. So overall time complexity is $O(n \log n)$.

Space Complexity: $O(1)$,We are not using any extra data structures apart from a few variables. The sorting is done in-place (or with constant space depending on language), so the space complexity is constant.